Lecture 2

Thursday, 23 July 2026 2:15 PM

Housekeeping items

- Lab quiz timings Wednesdays (as) : 1300 – 1700 hrs
- Penalty for academic dishonesty
  - Attendance: zero attendance will be recorded for the entire course
  - Unfair means during exams: DisCo + grade would be changed to D if receiving B- or higher, F otherwise
  - Plagiarism in lab assignments: zero for the entire lab component including lab quizzes
- Labs start on Monday (July 27)

Today's Lecture

1. Sorting algorithms (6)
2. Loop invariants to correctness of algorithms
3. Analyzing time

Sorting a list : Rearrange items in an "ascending" order

[2, 5, 4, 1, 3] → sorting algorithm → [1, 2, 3, 4, 5]

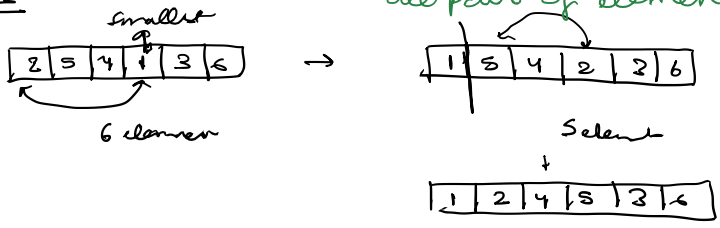
output

Sorting algos

Merge sort, Bubble sort, insertion sort, quick sort

[only possible if a[i] ≤ a[j] is defined for all pairs of elements]

Selection sort



Repeatedly

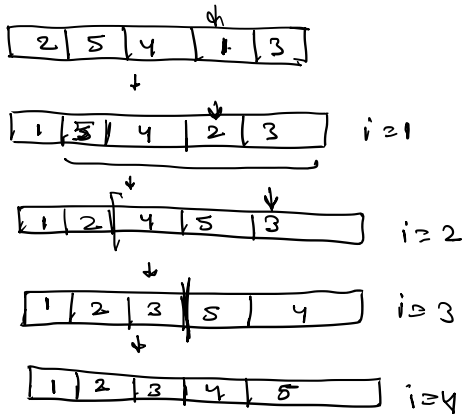
① Find the smallest element in the unsorted list

② Swap it with the first element of the unsorted list

Until no more elements in unsorted list

Program

```
def selection_sort(arr):
    n = len(arr)
    for i in range(n - 1):
        min_idx = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_idx]:
                min_idx = j
        #swap
        arr[i], arr[min_idx] = arr[min_idx], arr[i]
```



How do I prove correctness ?

Need to reason about the loop

Loop invariant : look for a statement that is true before every iteration of the loop

Initialization : does the statement start true

Maintenance : If stmt. is true before iteration $j-1$, then it is also true after iteration j

Termination : The final statement implies the desired statement to be true

Exercise 1

```
def find_minimum(a, start=0):
    """Return the index of the smallest element in a[start:]."""
    min_index = start
    for i in range(start + 1, len(a)):
        if a[i] < a[min_index]:
            min_index = i
    return min_index
```

Loop invariant : a[min_idx] is — smallest value in a[0,... i-1]

Initialization : i = 1 , min idx = 0

a[0] is smallest in a[0]

Maintenance :

To prove at $i+1$, $a[min\text{-}idx]$ is smallest in $a[0, \ldots, i]$

$min\text{-}idx^{i+1} = \min(a[i], a[min\text{-}idx^{i}])$

$\min(a[0 \ldots i-1])$

$= \min(a[0, \ldots i])$

Termination : $a[min\text{-}idx]$ is the smallest value value in $a[0, \ldots, n-1]$

Exercise 2

```
def selection_sort(a):
    """Sort the list a in place (ascending) using selection sort."""
    for i in range(len(a)):
        m = find_minimum(a, i)     # smallest element in the
unsorted part
        a[i], a[m] = a[m], a[i]    # swap it into position i
    return a
```

Loop invariant

→ Before iteration i, the $[0, \ldots, i-1]$ are sorted and also contain the i smallest values of the original list

Base case :

at $i = 0$

vacuously true

$P \Rightarrow Q$ is vacuously true if P is completely false

Maintenance

To prove

Before iteration $i+1$, $a[0, \ldots i]$ are sorted and also contain $i+1$ smallest values of a

clause 1 clause 2

↳ $a[i]$ is the smallest value in $[i, \ldots n-1]$

$a[i] \geq a[0, \ldots, i-1]$ → from iteration i

① clause 1 is true because already sorted and new entry added at the end

② clause 2 : $a[0 \ldots i+1]$ already ...

values

a[i] is smallest among [i, ... n-1]

Clearly a[0, ...i] are the i+1 smallest values among a

Termination: a[0, ...n-1] is sorted and contain the n smallest values of the list

→ There is a whole field on formal verification and reasoning of programs

Exercise: Write loop invariants for these programs

GCD

```
def func2(a, b):
    while b != 0:
        a, b = b, a % b
    return a
```

→ Write the loop invariant
→ Prove it hold

insertion sort

```
def func1(a):
    for i in range(1, len(a)):
        key = a[i]          # the element we want to insert
        j = i - 1
        # shift larger elements one slot to the right
        while j >= 0 and a[j] > key:
            a[j + 1] = a[j]
            j -= 1
        a[j + 1] = key      # drop key into the opening
    return a
```

3. How to reason about runtime complexity of selection sort or any algorithm?

Approach #1: Count the # of primitive steps

Approach #2: Experimental analysis

↳ Implement the algorithm

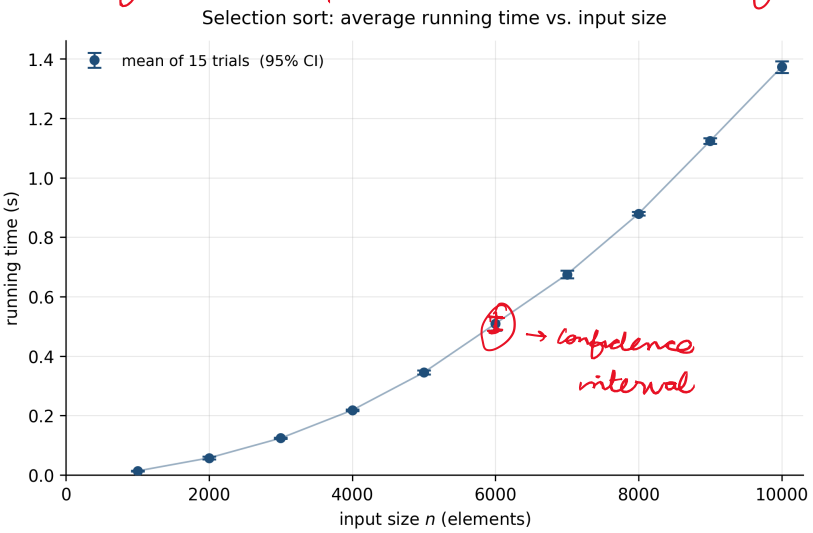
↳ Run it for n different inputs

↳ Report avg time

Run it and time it

```
import time, random
for n in [1000, 2000, 4000, 8000, 16000]:
    data = [random.random() for _ in range(n)]
    start = time.time()
    selection_sort(data)
    elapsed = time.time() - start
    print(n, elapsed)
```

Useful to report time as a f(input size)



Pitfalls

① Running time depends on the underlying machine → Programming Language

② You need to implement an algorithm → Implementation → H/W
to reason about its runtime

③ What inputs? → this may lead to difference
What about $n > 10{,}000$

④ To compare two algos: you need to execute them on same h/w and inputs

Approach #2: Count the # of primitive operations of an algorithm

What is a primitive operation? an operation so basic that for all practical purposes it takes a constant amount of time

Examples of primitive operations
- Assigning a value to a variable $a = 10$
- Performing an arithmetic operation $x + y$
- Comparing two numbers $x > y$
- Accessing an element of a list by index $a[i]$
- Calling a function or returning from one
;

Exercise 1: Count # of primitive operations

```
def find_minimum(a, start=0):
    """Return the index of the smallest element in a[start:]."""
    min_index = start      → c1
    n = len(a)             → c2
    i = start + 1          → c1
    while i < n:           → nc3
        if a[i] < a[min_index]:   → nc3
            min_index = i
        i += 1
    return min_index
```

should we count
① best case
② worst case → most useful as it gives worst case guarantee
③ average case → hard to compute

Principle
① count the # of primitive operations
② worst case analysis